AULA 11

VETORES

Arrays Unidimensionais em Linguagem C

Profª. Drª. Tatiele Bolson Moro

Ciência da Computação

AGENDA DA AULA

01

O Problema

Por que precisamos de vetores?

02

Conceito de Vetores

Arrays unidimensionais

03

Declaração

Sintaxe em linguagem C

04

Memória

Armazenamento contínuo

05

Manipulação

Leitura, escrita e acesso

06

Exemplos Práticos

Programas completos

O PROBLEMA

Como armazenar as notas de 1000 alunos?

✗ SEM VETORES

```
int nota1, nota2, nota3...
float nota999, nota1000;

→ Mil variáveis criadas
→ Código impossível de manter
```

✓ COM VETORES

```
float notas[1000];

→ Uma única variável!
→ Fácil de gerenciar
```

Variáveis Unitárias

Cada variável armazena somente UM valor de um tipo específico:

```
int nota1 = 8;
float media = 7.5;
char letra = 'A';
int total = 100;
```

Representação na memória:

nota1	media	letra	total
8	7.5	'A'	100

O que é um Vetor?

Um vetor (array unidimensional) é uma estrutura que permite armazenar vários valores do mesmo tipo sob um único nome de variável, acessados por índices numéricos.

Mesmo tipo: Todos os elementos devem ser do mesmo tipo de dado (int, float, char...)

Índices numéricos: Cada elemento é acessado por um índice, começando em 0

Memória contínua: Os elementos são armazenados em posições consecutivas na RAM

Declaração de Vetores

```
tipo  nome[tamanho];
```

tipo

int, float, char, double...

nome

identificador da variável

[tamanho]

quantidade de elementos

Exemplos de Declaração

```
int     meuVetor[100];
```

100 elementos inteiros

400 bytes na memória

```
float   meuOutroVetor[50];
```

50 números decimais

200 bytes na memória

```
char    maisUmVetor[25];
```

25 caracteres (string)

25 bytes na memória

Dica: Escolha nomes descritivos para seus vetores!

Memória RAM — Variáveis Comuns

Variáveis simples ficam em posições NÃO contínuas:

```
int a=5, b, c=7;
char d='a', e;
```

Posição	Variável	Valor
1	a	5
2		<lixo>
3		<lixo>
5	c	7
9	b	<lixo>
12	e	<lixo>
14	d	'a'

⚠ Posições NÃO são consecutivas!

Memória RAM — Vetores

Elementos de vetores são armazenados em posições CONTÍNUAS:

```
int vetA[8];
char vetB[5];
```

vetA[8]:

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
pos 1	pos 2	pos 3	pos 4	pos 5	pos 6	pos 7	pos 8

vetB[5]:

[0]	[1]	[2]	[3]	[4]
pos 9	pos 10	pos 11	pos 12	pos 13

✓ **Posições CONSECUTIVAS na memória!**

Índices do Vetor

Índice começa em ZERO
O primeiro elemento sempre tem índice 0

Último índice = tamanho - 1
Para int A[10], o último é A[9]

Exemplo: int A[10]

5	10	15	20	25	30	35	40	45	50
	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	

↑ índice 0 (primeiro)

↑ índice 9 (último)

Acessando Elementos

Use nome[índice] para ler ou escrever um elemento:

5	10	15	20	25	30	35	40	45	50
[0]	[1]	[2]		[4]	[5]	[6]	[7]	[8]	[9]

↑ A[3] = 20

```
// Atribuição:
A[0] = 10;          // coloca 10 no índice 0
A[5] = A[3] + 5;    // calcula e armazena

// Leitura:
printf("%d", A[3]); // imprime 20
```

Atribuição Passo a Passo

```
int b[5];
b[0] = 2;
b[2] = 5;
b[1] = b[0] - b[2];
b[3] = pow(b[2], 2);
b[4] = b[3] - 1;
```

Estado final do vetor b[5]:

2	-3	5	25	24
[0]	[1]	[2]	[3]	[4]

`b[0] = 2`	→ armazena 2 diretamente
`b[2] = 5`	→ armazena 5 diretamente
`b[1] = b[0]-b[2]`	→ 2 - 5 = -3
`b[3] = pow(5,2)`	→ $5^2 = 25$
`b[4] = b[3]-1`	→ 25 - 1 = 24

Laço FOR com Vetores

O laço for é a ferramenta ideal para percorrer vetores:

```
for(i = 0; i < 5; i++) {
    printf("%f\n", b[i]);
    //  i=0 → b[0], i=1 → b[1]...
}
```

Execução do laço:

i=0	i=1	i=2	i=3	i=4	i=5
b[0]	b[1]	b[2]	b[3]	b[4]	FIM

Exemplo 1 — Exibição de Valores

```c
#include <stdio.h>
#include <math.h>
int main(){
  int i;
  float b[5];
  b[0]=2; b[2]=5;
  b[1]=b[0]-b[2];
  b[3]=pow(b[2],2);
  b[4]=b[3]-1;
  for(i=0; i<5; i++){
    printf("%f\n", b[i]);
  }
}
```

Saída do Programa

```
2.000000
-3.000000
5.000000
25.000000
24.000000
```

Chave!

O índice i do for substitui o número fixo nos colchetes.

Exemplo 2 — Leitura Manual

```
#include <stdio.h>
int main(){
  int b[5], i;
    printf("Digite o valor 0: "); scanf("%d", &b[0]);
    printf("Digite o valor 1: "); scanf("%d", &b[1]);
    printf("Digite o valor 2: "); scanf("%d", &b[2]);
    printf("Digite o valor 3: "); scanf("%d", &b[3]);
    printf("Digite o valor 4: "); scanf("%d", &b[4]);

    for(i=0; i<5; i++){
        printf("posicao %d: %d\n", i, b[i]);
    }
}
```

⚠ Repetitivo! O próximo exemplo resolve isso com um for.

Exemplo 3 — Leitura com FOR

```c
#include <stdio.h>
//#include <math.h>
int main(){
  int b[5], i;
    // LEITURA:
    for(i=0; i<5; i++){
        printf("Valor %d: ", i);
        scanf("%d", &b[i]);
    }
    // EXIBIÇÃO:
    for(i=0; i<5; i++){
        printf("Posicao %d: %d\n",
        i, b[i]);
    }
}
```

Compacto
2 laços substituem 10 comandos

Escalável
Mudar de 5 para 1000 elementos? Só mude o 5!

Legível
Código mais limpo e fácil de entender

✓ **Versão muito mais elegante!**

Por que usar Vetores?

❌ **SEM Vetores — 50 alunos**

```
float nota1, nota2, nota3...
nota50;
printf("nota1: "); scanf("%f",
&nota1);
printf("nota2: "); scanf("%f",
&nota2);
...
printf("nota50: "); scanf(...);

media = nota1+nota2+...+nota50;
```

→ +200 linhas de código!

VS

✓ **COM Vetores — 50 alunos**

```
float notas[50], media=0;
int i;

for(i=0; i<50; i++){
  printf("Aluno %d: ", i);
  scanf("%f", &notas[i]);
  media += notas[i];
}
media = media/50;
```

→ Apenas 10 linhas!

Exemplo 4 — Média de 50 Alunos

```c
#include <stdio.h>
int main(){
  float notas[50], media=0;
    int i;
    for(i=0; i<50; i++){
        printf("Nota aluno %d: ", i);
        scanf("%f", &notas[i]);
        media += notas[i];
    }
    media = media / 50;
    printf("Media: %.2f\n", media);
}
```

`notas[50]`

vetor de 50 floats

`media += notas[i]`

acumula a soma no mesmo laço

`media / 50`

divide pelo total de alunos

`%.2f`

exibe com 2 casas decimais

Versão aprimorada com 1 loop!

Exemplo 5 — Maiores de 18 Anos

Ler idades de 40 alunos, contar e exibir os maiores de 18:

```
#include <stdio.h>
int main(){
    int idade[40], i, conta=0;
    for(i=0; i<40; i++){
        printf ("Informe a idade: ");
        scanf("%d", &idade[i]);
        if(idade[i] >= 18) conta++;
    }
    printf("Total: %d alunos\n", conta);
    for(i=0; i<40; i++){
        if(idade[i] >= 18)
        printf("Aluno %d: %d anos\n", i, idade[i]);
    }
}
```

Exemplo de saída

```
Total: 2 alunos
aluno 0 tem 28 anos
aluno 2 tem 19 anos
```

💡 Vetores permitem retornar ao início e reprocessar os dados!

Exemplo 6 — Maior Valor do Vetor

```c
#include <stdio.h>
int main(){
    int vetor[10], i, maior=-1000000;
    for(i=0; i<10; i++){
        printf("Valor %d: ", i);
        scanf("%d", &vetor[i]);
        if(vetor[i] > maior){
            maior = vetor[i];
        }
    }
    printf("Maior valor: %d", maior);
}
```

Como funciona:

1. **Inicializa maior** valor bem pequeno (-1000000)
2. **Lê cada elemento** e compara com maior
3. **Atualiza maior** se encontrar valor maior
4. **Exibe o resultado** após percorrer todo o vetor

```
Saída: O maior valor do vetor eh 10
```

Exemplo 7 — Dois Vetores (A e B)

Para cada valor de A: se par → multiplica por 5; se ímpar → soma 5 → armazena em B

```
#include <stdio.h>
int main(){
   int i, vetorA[15], vetorB[15];
    for(i=0; i<15; i++){
        scanf("%d", &vetorA[i]);
        if(vetorA[i] % 2 == 0)
            vetorB[i] = vetorA[i] * 5;
        else
            vetorB[i] = vetorA[i] + 5;
    }
    for(i=0; i<15; i++)
        printf("B[%d]=%d\n", i, vetorB[i]);
}
```

Exemplo de transformação:

A[i]	B[i]
7 (ímpar)	7+5=12
8 (par)	8×5=40
40 (par)	40×5=200
5 (ímpar)	5+5=10

🔑 Vetores distintos armazenam transformações de dados!

Exemplo 8 — Produto Escalar

Soma do produto de cada elemento de A pelo respectivo de B:

```
#include <stdio.h>
int main(){
    int vetorA[5]={7,8,9,1,5};
    int vetorB[5]={1,2,3,4,5};
    int prod=0, i;
    for(i=0; i<5; i++){
        prod += vetorA[i]*vetorB[i];
    }
    printf("Produto: %d", prod);
    // 7×1+8×2+9×3+1×4+5×5
    // = 7+16+27+4+25 = 79
}
```

Cálculo:

A[0]=7 × B[0]=1 = 7

A[1]=8 × B[1]=2 = 16

A[2]=9 × B[2]=3 = 27

A[3]=1 × B[3]=4 = 4

A[4]=5 × B[4]=5 = 25

Total: 7+16+27+4+25 = 79

⚠ Problema: Acesso Fora do Limite

Em C, não há verificação automática dos limites do vetor!

```
int vetor[5];
vetor[0]=0; vetor[1]=1;
vetor[2]=2; vetor[3]=3;
vetor[4]=4;

// ERROS - mas C não avisa!
vetor[5]  = 6;   // fora do limite!
vetor[-1] = 9;   // índice negativo!
vetor[10] = 12;  // muito além!
```

❌ Compilador não detecta

O código compila sem erros

⚡ Comportamento indefinido

Resultado imprevisível em execução

💥 Corrompimento de memória

Pode sobrescrever outras variáveis

✅ Solução: Sempre use o laço for com condição i < tamanho para garantir acesso seguro!

Boas Práticas com Vetores

1 Use #define para o tamanho

```
#define TAM 50
float notas[TAM];
```

2 Inicialize variáveis contadoras

```
int soma = 0; // SEMPRE
inicialize
```

3 Limite o for ao tamanho real

```
for(i=0; i<TAM; i++)
```

4 Use nomes descritivos

```
float temperaturas[30]; // bom!
float v[30]; // ruim!
```

scanf e printf com Vetores

scanf — Entrada de dados

```
scanf("%d", &vetor[i]); // inteiro
scanf("%f", &vetor[i]); // float
scanf("%s", vetor);     // char[] — sem & !
```

printf — Saída de dados

```
printf("%d", vetor[i]); // inteiro
printf("%f", vetor[i]); // float
printf("%s", vetor);    // string completa
```

⚠ Para char[], não use & no scanf! O nome do vetor já é um endereço.

Inicialização de Vetores

Inicialização explícita

```
int idades[5] = {18, 21, 17, 25, 22};
```

Tamanho automático

```
int primos[] = {2, 3, 5, 7, 11}; // tam=5
```

Zeros automáticos

```
float notas[10] = {0}; // todos zeros
```

Inicialização parcial

```
int v[5] = {1, 2}; // v[2],[3],[4] = 0
```

Vetor int A[5] = {18, 21, 17, 25, 22}:

18	21	17	25	22
[0]	[1]	[2]	[3]	[4]

Erros Comuns com Vetores

```
✗ vetor[tamanho]
✓ vetor[tamanho - 1]
```

Índice válido vai de 0 a tamanho-1

```
✗ for(i=1; i<=5; i++)
✓ for(i=0; i<5; i++)
```

Inicia em 0, termina antes do tamanho

```
✗ int v[5]; v[5]=10;
✓ v[4] = 10;
```

5 elementos: índices 0,1,2,3,4

```
✗ printf("%d", v);
✓ printf("%d", v[0]);
```

Sem índice imprime o endereço de memória

Quiz — Teste seus Conhecimentos!

1. Qual o índice do primeiro elemento de int v[10]?

a) 1

b) 0

c) -1

d) 10

2. Qual o último índice válido de float x[7]?

a) 7

b) 8

c) 6

d) 0

3. Para int A[5], qual comando lê o 3º elemento?

a) scanf("%d",&A[3])

b) scanf("%d",&A[2])

c) scanf("%d",&A[1])

d) scanf("%d",A)

Aplicações Práticas

Estatística

Calcular média, máximo, mínimo de datasets

Banco de Dados

Armazenar registros temporários em memória

Processamento de Sinais

Amostras de áudio, imagens pixel a pixel

Ordenação

Bubble Sort, Selection Sort sobre arrays

Jogos

Posições de objetos, inventário, tabuleiro

Texto

Processamento de strings e caracteres

Exercícios Propostos

01	Fácil	Declare um vetor de 10 inteiros, leia valores com for e exiba a soma de todos.
02	Médio	Leia 20 notas de alunos e exiba quantos foram aprovados (nota ≥ 7) e a média.
03	Médio	Crie dois vetores de tamanho 10. Some elemento a elemento e guarde em um 3º vetor.
04	Difícil	Encontre a posição (índice) do menor valor em um vetor de 15 floats.
05	Desafio	Leia 10 números. Inverta a ordem dos elementos e exiba o vetor invertido.

Resumo — Tudo que Aprendemos

Declaração

```
tipo
nome[tam];
```

Acesso

```
nome[indice]
```

Primeiro índice

```
sempre 0
```

Último índice

```
tamanho - 1
```

Leitura

```
scanf("%d",&v
[i])
```

Impressão

```
printf("%d",v
[i])
```

Percorrer

```
for(i=0;i<tam
;i++)
```

Memória

```
posições
contínuas
```

Conclusão!

- Vetores (arrays) armazenam múltiplos valores
- Declaração: tipo nome[tamanho]
- Índices começam em 0
- Use for para percorrer vetores
- Memória contínua e organizada
- Cuidado com acesso fora dos limites!

Bons estudos e bons programas! 🚀

Profª. Drª.
Tatiele Bolson Moro

Ciência da Computação